

Brain-Inspired Hyperdimensional Computing: How Thermal-Friendly for Edge Computing?

Paul R. Genssler¹, *Member, IEEE*, Austin Vas, and Hussam Amrouch², *Member, IEEE*

Abstract—Brain-inspired hyperdimensional computing (HDC) is an emerging machine learning (ML) method. It is based on large vectors of binary or bipolar symbols and a few simple mathematical operations. The promise of HDC is a highly efficient implementation for embedded systems, such as wearables. While fast implementations have been presented, other constraints have not been considered for edge computing. In this work, we aim at answering how thermal-friendly HDC for edge computing is. Devices, such as smartwatches, smart glasses, or even mobile systems have a restrictive cooling budget due to their limited volume. Although HDC operations are simple, the vectors are large, resulting in a high number of CPU operations and, thus, a heavy load on the entire system potentially causing temperature violations. In this work, the impact of HDC on the chip's temperature is investigated for the first time. We measure the temperature and power consumption of a commercial embedded system and compare HDC with the conventional convolutional neural network (CNN). We reveal that HDC causes up to 6.8°C higher temperatures and leads to up to 47% more CPU throttling. Even when both HDC and CNN aim for the same throughput (i.e., perform a similar number of classifications per second), HDC still causes higher on-chip temperatures due to the larger power consumption.

Index Terms—Convolutional neural network (CNN), edge computing, embedded system, hyperdimensional computing (HDC), temperature.

I. INTRODUCTION

WEARABLE devices are an emergent type of embedded system. Smartwatches, fitness trackers, e-textiles, or smart glasses are prominent examples. They detect seizures, measure the heart rate, categorize sport activities, augment reality to help the user navigate, or enable wireless payments, among others. Developers of such devices are trending toward embedded intelligence and smart things for more independence from phones and also for immediate feedback. The required machine learning (ML) and deep learning (DL) methods typically involve heavy computations and require relatively large storage for the models. However, in embedded systems, resources, such as processing power, memory, energy, and cooling are tightly constrained.

Brain-inspired hyperdimensional computing (HDC) has emerged as a promising alternative to the established ML

methods. Similar or better inference accuracy compared to a deep neural network (NN) has been demonstrated for speech recognition [1], EEG-based seizure detection [2], EMG-based gesture detection [3], or wafer map defect pattern classification [4], among others [5]. While layers of neurons are the base in NN, large vectors with dimensions in the thousands, hypervector, form the basic blocks in HDC. Their components can be simple bits, bipolar values, or more complex numbers and they are all independent of each other. Hence, parallelization is easy and, consequently, computationally efficient implementations have been proposed for ASICs [6], FPGAs [7], and embedded devices [2], [3].

Three basic operations, such as dot product, component-wise majority, and multiplication are used to encode the real-world data into a nonbinary hypervector. Compared to the floating-point matrix multiplications required by NN, such operations are lightweight, i.e., computationally less expensive. Therefore, HDC has been proposed as a promising ML method for embedded systems and smart things [1]–[3], [5]–[7]. Other advantages include the robustness against noise in input data, e.g., from unreliable cheap sensors, or in the model's memory as well as the ability to learn from very little data [2]. To realize the fast product cycle demanded by consumers, off-the-shelf hardware and software implementations are preferred by developers for their short time to market. Highly specialized hardware such as ASICs achieve better computational performance and higher energy efficiency. Nevertheless, HDC remains a demanding application due to the large size of the hypervector, data movement, and complex encoding.

In addition to memory, time, and power constraints face embedded systems, wearables in particular, also a temperature constraint. Such systems cannot be actively cooled because fans draw additional power, contain error-prone moving parts, and require space that is not available, e.g., in smart glasses. Passive cooling is limited on the one hand by the overall size of the system and, thus, its thermal capacity. On the other hand, if and how it has contact with the human skin can drastically limit the maximum sustainable temperature. The impact of HDC on the on-chip temperatures has not yet been evaluated.

Our Main Contribution: We are the first to investigate if HDC conforms with the thermal constraint that an embedded system imposes. If it causes temperature violations due to the demanding operations, then the promise as an alternative ML method for off-the-shelf embedded systems needs to be revisited. We employ an optimized state-of-the-art implementation of HDC [8] on a commercial Raspberry Pi 4 to classify images from the Fashion-MNIST data set [9], a common task, e.g., in smart glasses. We measure the temperature of the CPU and the power consumption of the whole system under realistic

Manuscript received 26 March 2022; revised 9 June 2022; accepted 6 July 2022. Date of publication 18 July 2022; date of current version 28 February 2023. This work was supported by Advantest as part of the Graduate School "Intelligent Methods for Test and Reliability" (GS-IMTR) at the University of Stuttgart. This manuscript was recommended for publication by F. Palumbo. (Corresponding author: Paul R. Genssler.)

The authors are with the Chair of Semiconductor Test and Reliability, University of Stuttgart, 70569 Stuttgart, Germany (e-mail: genssler@iti.uni-stuttgart.de; amrouch@iti.uni-stuttgart.de).

Digital Object Identifier 10.1109/LES.2022.3192093

workloads. HDC is compared against the convolutional neural network (CNN) during training and inference. We reveal that only HDC causes CPU throttling due to higher temperatures but still trains faster. In a scenario where both methods perform equally fast, HDC consumes more energy and causes higher on-chip temperatures.

II. PRELIMINARIES

The concepts of brain-inspired HDC can be implemented with different underlying vector symbolic architecture (VSA), e.g., with vectors of simple bits, bipolar values, integers, or real numbers. Depending on the type, different mathematical operations realize the basic operations of binding, bundling, permutation, and similarity score. In this work, real number hypervectors are employed because they offer the highest inference accuracy. A drawback is their higher computational cost compared to binary implementations.

To improve the inference accuracy, the concept of retraining has been proposed [1]. After an initial training cycle and class hypervector generation, the training data themselves are used for inference. If a sample is incorrectly classified, then the class hypervector is adjusted to be more similar to the sample. The process is repeated. Each iteration is referred to as an epoch.

Temperature can be a limiting factor in embedded devices. It correlates with the amount of processing performed by the compute units, which can be CPUs, GPUs, or other specialized ones, like tensor processing unit (TPU). In addition, memory accesses consume comparatively large amounts of electric energy, which translates again into thermal energy. Both sources of heat have to be controlled to achieve an overall lower system-wide temperature. Dynamic voltage and frequency scaling (DVFS) is one of the most important tools to manage the temperature. By reducing the clock frequency, the voltage can be reduced, having a quadratic impact on the dynamic power consumption.

III. EXPERIMENTAL SETUP

A Raspberry Pi 4 Model B with 8-GB RAM is employed in the experiments. The Raspberry Pi OS (32-bit Linux) uses kernel version 5.10.17-v7l and the firmware from 27 May, 2021. The nominal CPU frequency is 1500 MHz and starts throttling once its temperature exceeds 80 °C consistently (“soft max”). In the experiments, this throttling prevents the temperature from reaching the maximum of 85 °C. Temperature is measured with the on-chip sensor of the Broadcom 2711 system on chip (SoC) and, at idle, is on average 45 °C. The ambient temperature is kept constant at about 20 °C in all experiments for fair comparisons. The board is not covered, in a case, nor is a heat sink attached. Power measurements are done with an intermediary INA260 sensor between the USB power adapter output and the board, as shown in Fig. 1. The measurements are collected by the Pi itself via I²C. Idle power consumption is about 3.5 W with the CPU frequency throttled down to 600 MHz so save power.

Image classification is a demanding task for embedded systems. Each sample contains a lot of data that have to be processed. In this work, the Fashion-MNIST data set is employed as an example [9]. It comprises 60 000 training and 10 000 testing samples of ten different classes of clothing. All images are grayscale and have a resolution of 28×28 pixels.

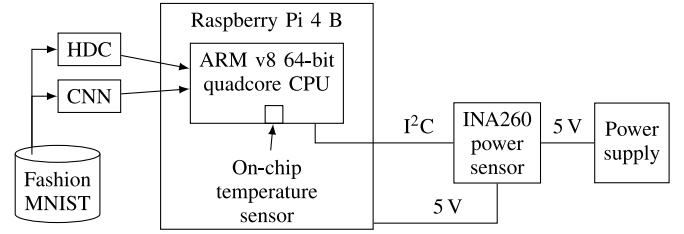


Fig. 1. Experimental setup with Fashion MNIST as a realistic workload.

TABLE I
CNN ARCHITECTURES EMPLOYED IN THE EXPERIMENTS

Layer type	Output shape	Parameters	Simplified CNN
Conv2D	(28, 28, 64)	320	(22, 22, 6)
MaxPooling2D	(14, 14, 64)	0	(11, 11, 6)
Dropout	(14, 14, 64)	0	(11, 11, 6)
Conv2D	(14, 14, 32)	8224	removed
MaxPooling2D	(7, 7, 32)	0	removed
Dropout	(7, 7, 32)	0	removed
Flatten	(1568)	0	(726)
Dense	(256)	401 664	(20)
Dropout	(256)	0	(20)
Dense	(10)	2570	(10)
Total trainable parameters		412 778	15050
Inference accuracy (70 epochs)		92.7 %	85.8 %

OnlineHD [8] is a state-of-the-art HDC implementation using the bipolar hypervector. Version 0.1.2 is built on top of Python’s PyTorch framework and provides efficient implementations for the underlying mathematical operations during encoding. The similarity computation is written in C++.

Inspired by the visual mechanism in the brain, CNNs were proposed as a special form of an NN with convolution layers, subsampling layers, and completely connected layers. Two CNN architectures are described in Table I and implemented with TensorFlow 1.14.0 and Python 3.7.3. The larger CNN is used in the experiments, while the simplified CNN is discussed in Section IV-C. Neither model utilizes the GPU during training or inference, all processing is done on the CPU.

IV. EXPERIMENTAL RESULTS

All tasks, such as training and inference, are continuously repeated for 4 h. This allows the system to accumulate heat and to minimize environmental influences. The results presented in the figures are short representative segments from these long experiments. The reported numbers are averaged over the whole experiment.

A. Inference Accuracy

Each model, CNN and HDC, is trained on a server machine with no memory, temperature, or power constraint. Training the models longer does not increase the energy consumption or the thermal emissions of the deployed model. During training, only the neurons’ parameters or the hypervector are changed, not the CNN architecture itself or the dimension. Therefore, fully trained models can be deployed to the target system. After training for 70 epochs, the CNN achieves an inference accuracy of 92.7 % on the test set. For HDC, the various dimensions are explored and summarized in Table II, from 500 up to 10 000 achieving an inference accuracy of 82.3 %

TABLE II
INFERENCE ACCURACY FOR DIFFERENT DIMENSIONS WITH HDC [8], A NINE DEGREE POLYNOMIAL SVM, LOGISTIC REGRESSION (LR), AND THE CNN FOR COMPARISON

Dimension	500	4000	10000	SVM	LR	CNN
ISOLET	92.2	93.4	93.6	96.3	96.0	—
MNIST	91.2	94.8	95.1	98.4	92.6	99.2
FMNIST	82.3	86.0	86.7	89.4	84.3	92.7

and 86.7 %, respectively. Even though this accuracy increases with the dimension, anything above 4000 (86.0 %) would have a diminishing return for an embedded system. It increases energy consumption and inference latency. Hence, a dimension of 4000 is selected for the remaining experiments.

B. Training Analysis on the Embedded System

The training of an ML model involves lots of data and computations. Hence, it is usually performed in the cloud and only the fully trained model is deployed to the target system. However, for personalized applications, such as seizure detection or voice recognition, the model could be trained further to improve inference performance with the user. In this experiment, the model was trained from scratch to evaluate the overall impact of this type of workload on temperature. The data set is loaded once into memory and the training is repeated to simulate a larger data set. Re/training epochs are reduced to 1 and 20, resulting in a reduced inference accuracy of 86.0 % and 81.9 % for the CNN and HDC, respectively. The CNN does not exceed 79 °C with 75.6 °C on average as shown in Fig. 2. In contrast, HDC reaches temperatures of up to 84.2 °C, 6.7 °C more than CNN on average. HDC is frequently close to the maximum temperature of 85 °C during the encoding of the data set. Hence, DVFS is employed by the SoC, reducing the clock frequency up to 47 % (800 MHz). During retraining, the temperature drops to about 79 °C. No thermal buffers are created for the next encoding cycle. The power consumption is similar for both models with 5.2 W on average. Therefore, the operations used by HDC cause more heat compared to the CNN. Yet, despite the CPU throttling, HDC completes a training cycle 14.9 % faster as listed in Table III. However, inference accuracy on the test set is 4 % lower with HDC.

C. Continuous Inference Analysis

The primary task of the ML model in an embedded system is the inference of incoming samples. Smart glasses could analyze the live camera feed to overlay directions for an augmented reality experience. A smartphone listens for keywords to activate the voice assistant. To mimic a similar scenario, the inference of the test data set is continuously repeated.

HDC processes 1000.5 images/s, which is $3.1\times$ more than CNN because of its less complex operations. It also utilizes the CPU to its full extent, whereas the CPU load during CNN inference is only about 75 %. However, because of the more complex operations, the CNN consumes 5.87 W compared to HDC with 5.44 W, on average. Although the power consumption is 8 % higher with a CNN, the temperature is 2.1 °C lower on average. Furthermore, the CNN exceeds the first temperature limit of 80 °C for 7.2 % of the total time compared to

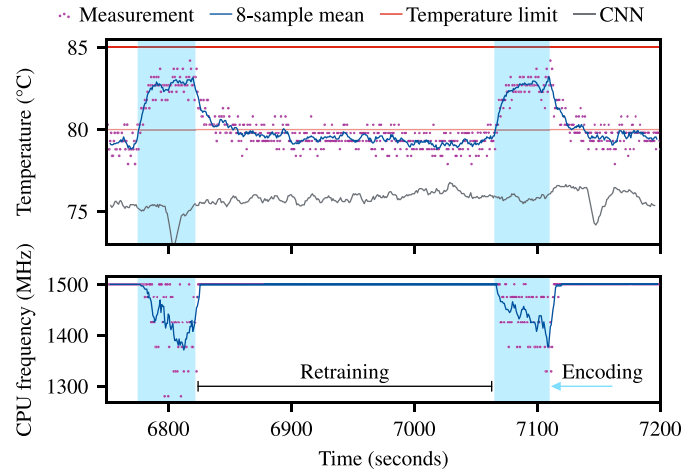


Fig. 2. Encoding during repeated training in HDC causes CPU throttling. Despite that, HDC is still faster in training the model.

TABLE III
AVERAGE NUMBER OF IMAGES PROCESSED PER SECOND

Scenario	Training	Inference	Inference
Temperature Limit	85 °C	85 °C	60 °C
HDC	206.3	1000.5	382.0
CNN	179.5	322.8	373.0

47.5 % with HDC. The trend in the data for CNN suggests an increase in temperature above 80 °C if the data set was larger. However, the setup time between two inference runs reduces CPU load and temperature. During this setup period, book-keeping tasks are done and the inference accuracy is calculated. The same applies to HDC, although the temperature increases faster and could reach 85 °C, triggering DVFS. The measured CPU clock frequency reduction is not because of over-temperature but enabled by the reduced CPU load during setup.

A simplified CNN has been tested as well, its structure is described in Table I. Due to its reduced complexity, the number of processed images increases to 2427.5 outperforming HDC. The power consumption remains at the level of 5.3 W.

D. Inference Analysis Under Temperature Constraint

A Raspberry Pi has an upper temperature limit of 85 °C, an integrated heat spreader made of metal, and a large PCB to dissipate the thermal energy. Smaller edge devices, embedded systems, and wearables do not have such high temperature limits or thermal mass. To simulate these tighter temperature constraints with a Raspberry Pi, its clock frequency is throttled if a temperature threshold is crossed. In this experiment, the threshold is set to 60 °C. If the devices touch the skin, such high temperatures could harm the user. A small script checks the temperature every second and sets the CPU frequency to 600 MHz in the case of overheating. As shown in Fig. 3, both models exceed the limit even with a throttled CPU clock. The number of processed images is reduced from 1001 (no limit, Section IV-C) to 382 for HDC and increases for CNN from 323 to 373. HDC has a 22 % higher CPU load and consumes 7 % more energy. This results in a 2 °C higher temperature on average.

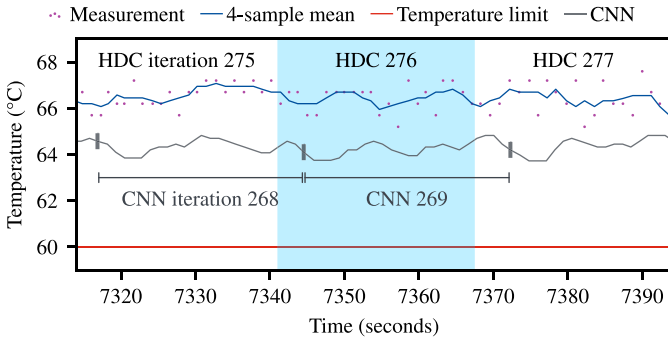


Fig. 3. Inference with a temperature constraint of 60°C. Both models cause permanent CPU throttling to 600 MHz. In addition, the temperature with HDC is 2°C higher but both classify a similar number of images per second.

E. Handcrafted Encoding for ISOLET

To provide an additional perspective, the ISOLET data set [10] of voice samples is analyzed. The first model is OnlineHD, which achieves an accuracy of 94.7% with a dimension of 2048. The second model is based on the C-Code implementation of HD-Lib,¹ uses binary vectors of dimension 4096 and a record encoding to achieve an accuracy of 87.1% (without retraining). The third model is a KNN classifier with $n = 9$ and 92.1% accuracy implemented with the Scikit-learn library.

OnlineHD, HD-Lib, and KNN process 2369.5, 537.7, and 390.1 samples/s, respectively. OnlineHD also consumes the most energy (5.9 W) and generates the most heat (82.8°C) reducing the clock frequency to 1405 MHz on average. The HD-Lib-based handcrafted encoding consumes slightly less power (5.7 W) and, consequently, generates less heat (82.1°C) but still throttles the CPU to 1431 MHz on average. The KNN does not utilize all CPU cores fully and the average temperature is about 74.9°C, not throttling the CPU and consuming less power (5.5 W).

V. DISCUSSION

The following conclusions can be drawn:

- 1) HDC performs training and inference faster than CNN. However, under temperature constraints and, thus, reduced CPU frequency because of throttling, the throughput (images per second) is reduced more with HDC compared to CNN. The number of images processed per second decreases by 33% and 62% during training and inference, respectively.
- 2) In contrast, with CNN, the throughput decreases 16% and even increases 16% during training and inference, respectively. These results suggest HDC is *compute bound* since a reduction in frequency by 60% reduces the inference time by 62%. The CNN, on the contrary, is *memory bound*. The CPU is stalled waiting for data. A reduced frequency reduces this wait time and the CPU utilization increases.
- 3) Nevertheless, HDC consistently causes higher temperatures than CNN. During inference, HDC exceeded the targeted 80°C almost half of the time, 6.6× more often

than CNN, yet HDC performs $3.1\times$ more inferences per second. However, even with reduced CPU frequency, HDC exceeds the temperature limit by 2°C more than CNN, while both have comparable inference speed. Because of its high number of computations compared to CNN, HDC performs worse regarding temperature and power consumption in such low-power scenarios expected for wearable devices.

- 4) The efficiency of the CNN is improved by using a 64-bit OS. The number of images processed per second jumps to 747 while the power consumption is reduced to 5.0 W at the same time. Alternatively, quantizing the CNN to 8-bit integers doubles the processing speed. A simplified CNN, described in Table I, achieves after 70 epochs training the same accuracy as HDC but processes 2427.5 images/s at 5.3 W, although at 77.7°C. Handcrafted HDC encoding on a different data set suggests a similar pattern. With off-the-shelf hardware, HDC is outperformed by other methods in nonfunctional metrics, such as power consumption and temperature.

VI. CONCLUSION

This was the first work to investigate the impact of HDC on the on-chip temperature of a CPU. We conducted several experiments on a commercial embedded system. Power consumption and temperature were measured while executing HDC and CNN algorithms for both training and inference. We unveiled that HDC faces a temperature and power challenge in low-power systems such as wearables and, hence, it is less thermal-friendly in edge computing.

REFERENCES

- [1] M. Imani, D. Kong, A. Rahimi, and T. Rosing, "VoiceHD: Hyperdimensional computing for efficient speech recognition," in *Proc. IEEE Int. Conf. Reboot. Comput. (ICRC)*, 2017, pp. 1–8.
- [2] A. Burrello, S. Benatti, K. Schindler, L. Benini, and A. Rahimi, "An ensemble of hyperdimensional classifiers: Hardware-friendly short-latency seizure detection with automatic iieeg electrode selection," *IEEE J. Biomed. Health Inform.*, vol. 25, no. 4, pp. 935–946, Apr. 2021.
- [3] A. Moin *et al.*, "A wearable biosensing system with in-sensor adaptive machine learning for hand gesture recognition," *Nat. Electron.*, vol. 4, no. 1, pp. 54–63, Dec. 2020.
- [4] P. R. Genssler and H. Amrouch, "Brain-inspired computing for wafer map defect pattern classification," in *Proc. IEEE Int. Test Conf. (ITC)*, 2021, pp. 123–132.
- [5] L. Ge and K. K. Parhi, "Classification using hyperdimensional computing: A review," *IEEE Circuits Syst. Mag.*, vol. 20, no. 2, pp. 30–47, 2nd Quart., 2020.
- [6] F. Montagna, A. Rahimi, S. Benatti, D. Rossi, and L. Benini, "PULP-HD: Accelerating brain-inspired high-dimensional computing on a parallel ultra-low power platform," in *Proc. 55th ACM/ESDA/IEEE Design Autom. Conf. (DAC)*, 2018, pp. 1–6.
- [7] M. Schmuck, L. Benini, and A. Rahimi, "Hardware optimizations of dense binary hyperdimensional computing: Rematerialization of hyper-vectors, binarized bundling, and combinational associative memory," *J. Emerg. Technol. Comput. Syst.*, vol. 15, no. 4, p. 32, Oct. 2019.
- [8] A. Hernandez-Cane, N. Matsumoto, E. Ping, and M. Imani, "OnlineHD: Robust, efficient, and single-pass online learning using hyperdimensional system," in *Proc. Design Autom. Test Europe Conf. Exhibition (DATE)*, 2021, pp. 56–61.
- [9] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.
- [10] D. Dua and C. Graff, "UCI Machine Learning Repository," 2017. [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/ISOLET>

¹<https://github.com/skurella/hdlib>